JASHORE UNIVERSITY OF SCIENCE & TECHNOLOGY

Department of Computer Science & Engineering

Course: Artificial Intelligence & Machine Learning Lab • Course Code: CSE-3202

CNN-Based Fruit Image Classification Using PyTorch

Standard Dataset Training and Real-World Smartphone Image Evaluation

A Project Report Submitted in Partial Fulfilment of the Requirements for the Course

Submitted By

Name: Mehedi Hasan Sagor

Student ID: 210137

Department: Computer Science & Engineering

Submitted To

Course Teacher: Dr. A F M Shahab Uddin

Designation: Assistant Professor

Department: Computer Science & Engineering

Submission Date: 17/08/2026

Table of Contents

Abstract.... 4
1. Introduction.... 4
2. Problem Statement.... 6
3. Project Objectives.... 7
4. Dataset Description.... 7
5. Dataset Distribution Analysis.... 9
6. Custom Real-World Dataset.... 9
7. Data Preprocessing.... 10
8. Complete Image Processing Pipeline.... 12
9. DataLoader Design.... 12
10. CNN Architecture.... 13
11. Convolution Mathematics.... 14
12. Activation Function.... 15
13. Max Pooling.... 15
14. Fully Connected Classifier.... 15
15. Dropout.... 16
16. Output Layer.... 16
17. Softmax.... 16
18. Loss Function.... 17
19. Optimizer.... 17
20. Backpropagation and Training Loop.... 18
21. Hyperparameters.... 18
22. Hyperparameter Configuration and Recommended Tuning Strategy.... 19
23. Model Training.... 20
24. Training History Analysis.... 20
25. Overfitting and Underfitting Analysis.... 21
26. Model Evaluation.... 22
27. Precision.... 22
28. Recall.... 23
29. F1-Score.... 23
30. Confusion Matrix.... 23
31. Error Analysis.... 24
32. Real-World Smartphone Evaluation.... 24
33. Custom Image Prediction.... 25

34. Confidence Analysis..... 26
35. Standard Dataset vs. Real-World Performance..... 26
36. Complete System Architecture..... 27
37. PyTorch Implementation..... 27
38. Model Saving and Reproducibility..... 29
39. Automation Workflow..... 30
40. Computational Complexity..... 30
41. Limitations..... 31
42. Future Work..... 32
43. Conclusion..... 32
44. References..... 33
45. Appendix..... 33
Report Verification Summary..... 34

Abstract

Automated fruit recognition has practical value for retail self-checkout, agricultural sorting, and dietary-tracking applications, yet models trained on clean benchmark datasets are not guaranteed to work when confronted with the uncontrolled photographs an end user actually takes. This project addresses that gap by training a custom Convolutional Neural Network (CNN) in PyTorch to classify fruit images into three categories — Apple, Banana, and Orange — and then explicitly measuring how well the trained model transfers to ten independently captured smartphone photographs of real fruit.

The model was trained on the Fruits-360 dataset, from which all Apple, Banana, and Orange sub-varieties were merged into three target classes, yielding a training pool of 8,313 images and a held-out test split of 2,778 images. After an internal 90/10 train–validation split, 7,481 images were used for training and 832 for validation. Each image was resized to 128×128 pixels, converted to a tensor, and normalized with mean and standard deviation of 0.5 per channel; training images were further augmented using random resized cropping, horizontal flipping, rotation, affine translation/shear, colour jitter, and random erasing.

The CNN consists of four convolutional blocks (32, 64, 128, and 128 filters) each followed by batch normalization and ReLU, with max-pooling after the first three blocks and adaptive average pooling before a fully connected classifier of 2,048→256→3 units with a dropout of 0.5. The network was trained with the Adam optimizer (learning rate 0.001) and cross-entropy loss, using a ReduceLROnPlateau scheduler and early stopping; training halted at epoch 8 of a maximum 25, and the checkpoint with the best validation accuracy (98.56%, achieved at epoch 2) was retained as the final model.

On the standard test split, the model achieved a test accuracy of 97.84% and a test loss of 0.0620, with near-perfect precision and recall for Apple and Orange and a lower Banana precision of 0.89 due to occasional Apple→Banana confusion. On the ten real-world smartphone images — preprocessed with automated background removal before inference — the model correctly classified 6 of 10 images (60% accuracy), with every misclassification affecting the Orange class, which is also the most under-represented class in training (5.76% of the training pool). The results confirm strong in-distribution performance alongside a substantial real-world generalization gap, underscoring the importance of dataset class balance and domain-shift-aware evaluation for deployment-ready fruit classification systems.

1. Introduction

Image classification is a fundamental computer vision task in which a model is trained to assign a category label to an input image from a predefined set of classes. Formally, given an image represented as a tensor of pixel intensities, the goal is to learn a mapping from the pixel space to a discrete label space. This task underlies a wide range of applications, including object recognition, medical imaging, autonomous navigation, and agricultural automation, and forms the basis of the classification system developed in this project.

Supervised learning is the paradigm in which a model is trained on a labelled dataset, that is, a collection of input–output pairs (image, class label). During training, the model adjusts its internal parameters to minimize the discrepancy between its predictions and the ground-truth labels, so that it can generalize to unseen examples at test time. This project follows a strictly supervised approach: every training and validation image is associated with a known fruit category.

Convolutional Neural Networks (CNNs) are particularly well suited to image classification because they exploit the spatial structure of images through local receptive fields, weight sharing, and hierarchical feature extraction. Unlike fully connected networks applied directly to raw pixels, convolutional layers preserve spatial locality and drastically reduce the number of trainable parameters, while pooling layers introduce a degree of translation invariance. Successive convolutional blocks learn increasingly abstract representations, progressing from edges and textures in early layers to shape and object-level features in deeper layers.

The motivation for fruit classification stems from its relevance to automated agricultural sorting, retail self-checkout systems, dietary tracking applications, and general-purpose object recognition research. Fruit categories such as Apple, Banana, and Orange present a tractable but non-trivial classification problem: the classes are visually distinct in colour and shape, yet subject to considerable intra-class variation due to ripeness, angle, and lighting.

A key distinction in this project is between classification performance on standard, curated datasets and performance on real-world images. Standard datasets are typically collected under controlled conditions — uniform backgrounds, consistent lighting, and centred objects — which allows models to achieve high accuracy but does not guarantee that this performance transfers to uncontrolled, real-world settings. Real-world images captured with consumer devices such as smartphones introduce additional variability that a model trained purely on curated data may not have encountered.

Testing the trained model on smartphone photographs is therefore an essential component of this project. It provides an empirical measure of the model's ability to generalize beyond its training distribution and exposes practical failure modes that would remain hidden if evaluation were restricted to the standard test split. This form of evaluation is directly relevant to any deployment scenario in which the model would ultimately process images captured by ordinary users rather than curated benchmark images.

From a practical standpoint, this project demonstrates a complete, reproducible deep learning pipeline — from raw dataset acquisition through model training, evaluation, and deployment-style inference on novel images — implemented using PyTorch and made fully reproducible through a GitHub repository executed in Google Colab. This mirrors the workflow expected in applied machine learning engineering, where model robustness under real-world conditions is as important as headline accuracy figures obtained on a held-out test set drawn from the same distribution as the training data.

1.1 The Research and Engineering Problem

The central engineering problem addressed in this project is twofold: (1) to design, implement, and train a CNN capable of accurately distinguishing between Apple, Banana, and Orange images drawn from a standard dataset, and (2) to rigorously assess whether this trained model generalizes to real-world smartphone photographs of the same fruit categories, which were not part of the training or validation distribution.

1.2 Domain Shift: Training-Domain Images vs. Real-World Images

A model trained exclusively on a curated dataset implicitly learns the statistical characteristics of that dataset's image distribution. When the model is subsequently applied to images drawn from a different distribution — such as smartphone photographs taken in uncontrolled environments — a discrepancy known as domain shift may degrade performance. Several factors commonly contribute to this shift:

- Lighting variation — differences in illumination intensity, colour temperature, and shadow patterns between the training images and real-world photographs.
- Background variation — standard datasets often use plain or white backgrounds, whereas smartphone photographs may include cluttered, textured, or naturally coloured backgrounds.
- Rotation — real-world objects may be photographed at arbitrary orientations not well represented in the training set.
- Scale — the apparent size of the fruit in the frame can vary substantially depending on camera distance.
- Camera quality — differences in sensor resolution, compression artefacts, and colour reproduction between the dataset source and the smartphone camera.
- Object position — fruits may not be centred or fully visible within the frame.
- Image noise — real-world images may contain sensor noise, motion blur, or focus artefacts absent from curated datasets.
- Dataset bias — the standard dataset may over-represent particular fruit varieties, colours, or photographic conditions relative to the real world.
- Domain shift — the cumulative effect of the above factors, formally described as a mismatch between the training distribution P_train(X) and the real-world distribution P_real(X).

This project treats domain shift not as a peripheral concern but as a primary evaluation axis, explicitly comparing standard test-set performance against performance on ten independently captured smartphone photographs.

2. Problem Statement

The classification task is formulated as a supervised, multiclass image classification problem. The input is defined as an RGB image tensor:

$$X \in \mathbb{R}^{H \times W \times 3}$$

H, W — the height and width of the input image in pixels

3 — the three colour channels (Red, Green, Blue)

The model must map each input image to one of three mutually exclusive fruit categories:

$$Y \in \{ Apple, Banana, Orange \}$$

The classification task performed by the CNN is expressed mathematically as:

$$f_\theta(X) \rightarrow Y$$

X — the input image

f_θ — the convolutional neural network, parameterized by the weight set θ

Y — the predicted fruit class

The learning objective is to determine the parameter set θ that minimizes the expected classification error over the training distribution, formally expressed as minimizing a loss function L(f_θ(X), Y_true) averaged over the training dataset (see Section 18, Loss Function). The trained model is subsequently evaluated on data drawn from two distinct distributions: the held-out standard test split, and the independently

collected smartphone image set, in order to assess both in-distribution accuracy and out-of-distribution generalization.

3. Project Objectives

- Build an end-to-end CNN classification pipeline covering data acquisition, preprocessing, training, evaluation, and inference.
- Automatically obtain the standard fruit image dataset as part of a reproducible, scripted workflow.
- Select the Apple, Banana, and Orange classes from the full set of classes available in the source dataset.
- Preprocess all images into a consistent tensor format suitable for CNN input.
- Design and implement a custom CNN architecture using PyTorch.
- Train the model on the training split while monitoring performance on a validation split.
- Validate the model to guide model selection and detect overfitting or underfitting.
- Evaluate the trained model on an unseen standard test split.
- Generate a confusion matrix summarizing class-level performance on the test set.
- Perform qualitative error analysis on misclassified test images.
- Save the trained model as a PyTorch state dictionary (.pth file) for reproducibility and reuse.
- Automatically load custom smartphone images from a GitHub repository as part of the reproducible pipeline.
- Test the trained model using ten real-world smartphone photographs not present in the training or validation data.
- Calculate the model's prediction confidence for each custom image using the softmax output.
- Ensure full reproducibility of the project through a combination of a GitHub repository and execution in Google Colab.

4. Dataset Description

This section documents the standard image dataset used for training, validation, and testing, together with the composition of the three selected classes.

4.1 Dataset Summary

Field	Value
Dataset name	Fruits-360
Dataset source	GitHub — Horea94/Fruit-Images-Dataset (downloaded automatically as a ZIP archive)
Dataset purpose	Benchmark dataset for fruit image classification

Image format	JPG, organized in per-variety class folders under Training/ and Test/
Classes originally available	130+ fruit/vegetable classes (including many sub-varieties of apple, banana, and orange)
Classes actually used	3 merged classes: Apple, Banana, Orange
Sub-varieties merged into "Apple"	Apple Red Delicious, Apple Pink Lady, Apple Red Yellow 1/2, Apple Braeburn, Apple Red 1/2/3, Apple Golden 1/2/3, Apple Crimson Snow, Apple Granny Smith (13 sub-varieties)
Sub-varieties merged into "Banana"	Banana, Banana Red, Banana Lady Finger (3 sub-varieties)
Sub-varieties merged into "Orange"	Orange (1 sub-variety only)
Image dimensions after preprocessing	128 × 128 pixels, RGB
Colour characteristics	RGB (3 channels)

4.2 Dataset Split Table

The images belonging to the merged Apple, Banana, and Orange classes were copied out of the Fruits-360 Training/ and Test/ folders into a new three-class dataset. Table 1 reports the actual per-class image counts obtained directly from this copy step and from the subsequent 90/10 train–validation split of the training pool.

Class	Training Pool	Standard Test	Total
Apple	6,404	2,134	8,538
Banana	1,430	484	1,914
Orange	479	160	639
Total	8,313	2,778	11,091

Table 1. Per-class image counts in the training pool and the standard test split (source: notebook Cell 9 output).

The 8,313-image training pool was further split 90/10 into a training subset and a validation subset using PyTorch's random_split with a fixed random seed, yielding 7,481 training images and 832 validation images overall. The notebook does not print the per-class breakdown of this particular 90/10 split, so the exact per-class training-versus-validation counts are not reported in the supplied project materials; only the aggregate split sizes and the pre-split per-class distribution shown in Table 1 are available.

Split	Number of Images
Training (used for gradient updates)	7,481
Validation (used for model selection)	832
Standard test (held out, never seen in training)	2,778

Table 2. Overall dataset split sizes (source: notebook Cell 15 output).

5. Dataset Distribution Analysis

Class distribution is defined as the proportion of the dataset occupied by each class:

$$P(C\_i) = N\_i / N$$

- ***N_i*** — number of samples belonging to class i
- ***N*** — total number of samples in the dataset

Applying this formula to the 8,313-image training pool (Table 1) gives the following actual class distribution:

Class	Training Pool Count	P(C_i)
Apple	6,404	77.04%
Banana	1,430	17.20%
Orange	479	5.76%

Table 3. Class distribution of the training pool, computed from Table 1.

The dataset is clearly imbalanced: Apple alone accounts for over three-quarters of the training pool, while Orange — represented by only a single Fruits-360 sub-variety — makes up less than 6%. The standard test split exhibits an almost identical imbalance (Apple 76.82%, Banana 17.42%, Orange 5.76%), so test accuracy is not strongly distorted by a train/test distribution mismatch; however, this shared imbalance means a model could score a high overall accuracy while performing poorly on the minority Orange class, since Orange contributes only a small share of both the training loss signal and the overall accuracy calculation.

The actual implementation applies no class weighting: the loss function is a plain nn.CrossEntropyLoss() with no weight argument (Section 19), and no oversampling or undersampling of the minority classes is used. Despite this, the model still achieved perfect precision and recall on Orange on the standard test set (Section 26), suggesting Orange images are visually distinctive enough for the CNN to separate cleanly even with few examples. This in-distribution robustness did not carry over to the real-world smartphone evaluation, however, where every misclassification involved the Orange class (Section 33) — showing that accuracy alone, computed on an imbalanced, in-distribution test set, understated the model's fragility on the minority class once the input distribution changed. This is the main motivation for reporting per-class precision, recall, and F1-score (Sections 27–29) rather than relying on overall accuracy.

6. Custom Real-World Dataset

To evaluate generalization beyond the standard dataset's distribution, ten photographs of real fruits were captured using a smartphone camera and committed to the project's GitHub repository under datasets/. The actual class composition is four Apple images, three Banana images, and three Orange images, matching the filename prefix of each file.

Image	Actual Class	Image Source	Environment
apple1.jpeg	Apple	Smartphone	Real-world
apple2.jpeg	Apple	Smartphone	Real-world
apple3.jpeg	Apple	Smartphone	Real-world
apple4.jpeg	Apple	Smartphone	Real-world
banana1.jpeg	Banana	Smartphone	Real-world
banana2.jpeg	Banana	Smartphone	Real-world
banana3.jpeg	Banana	Smartphone	Real-world
orange1.jpeg	Orange	Smartphone	Real-world
orange2.jpeg	Orange	Smartphone	Real-world
orange3.jpeg	Orange	Smartphone	Real-world

Table 4. Composition of the custom smartphone image set (source: notebook Cell 5 output).

Real fruits were photographed independently, rather than sourced from the standard dataset, to guarantee that the custom evaluation set is entirely disjoint from the training distribution. These images were never used for training or validation, so they serve as a genuine out-of-distribution test of the model's generalization capability. Because the photographs are captured under uncontrolled lighting, natural backgrounds (including a hand holding the fruit in several images), and a consumer smartphone camera, they represent a substantially stronger and more realistic test of deployment-readiness than the standard test split. As shown in Section 35, their inclusion reveals a real-world accuracy far below the standard test accuracy, directly quantifying the effect of domain shift.

7. Data Preprocessing

7.1 Image Loading

Images are loaded from disk using the Python Imaging Library (PIL) or OpenCV. PIL's Image.open() reads compressed image files (e.g., JPEG, PNG) and decodes them into an in-memory array of pixel values, which torchvision transforms subsequently operate on. OpenCV provides an alternative loading path but represents images in BGR channel order by default, requiring explicit conversion to RGB.

7.2 RGB Conversion

All images are converted to the RGB colour space to ensure a consistent three-channel representation across the dataset, since some source images may be stored in grayscale, RGBA, or CMYK formats. Standardizing to RGB guarantees that every input tensor has the same channel structure expected by the CNN's first convolutional layer.

7.3 Image Resizing

All images are resized to a fixed spatial resolution of 128 × 128 pixels (IMAGE_SIZE = 128 in the notebook). CNNs with fully connected classification heads require a fixed-size input tensor; variable input dimensions would produce a variable-length flattened feature vector incompatible with the fully connected layer's fixed weight matrix. During training, resizing is combined with RandomResizedCrop (Section 7.6); during validation, testing, and custom-image inference, a plain deterministic Resize((128, 128)) is used.

7.4 Tensor Conversion

The torchvision transform ToTensor() converts a PIL image with pixel values in the range [0, 255] into a PyTorch tensor with values rescaled to the range [0, 1], and permutes the array into PyTorch's channel-first layout:

$$X \in \mathbb{R}^{3 \times H \times W}, \quad [0, 255] \rightarrow [0, 1]$$

7.5 Normalization

The notebook normalizes each channel using MEAN = [0.5, 0.5, 0.5] and STD = [0.5, 0.5, 0.5], rather than ImageNet statistics. Each channel is standardized according to:

$$x' = (x - \mu) / \sigma$$

With μ = σ = 0.5 per channel, this transformation maps the [0, 1] pixel range produced by ToTensor() into the symmetric range [−1, 1]. This centres the pixel distribution around zero, which stabilizes gradient magnitudes during training and generally accelerates convergence; ImageNet-specific statistics were not used since the model is trained from scratch rather than fine-tuned from an ImageNet-pretrained backbone.

7.6 Data Augmentation

The following augmentation transforms are actually applied to the training split (train_transform in the notebook), in this order:

- **RandomResizedCrop(128, scale=(0.7, 1.0)) —** randomly crops between 70% and 100% of the image area and resizes it to 128×128, varying the apparent scale and framing of the fruit.
- **RandomHorizontalFlip() —** flips the image left-to-right with 50% probability, since fruit orientation is not a distinguishing feature of its class.
- **RandomRotation(25) —** rotates the image by a random angle up to ±25°, simulating arbitrary camera/object orientation.
- **RandomAffine(degrees=0, translate=(0.1, 0.1), shear=5) —** randomly translates the image by up to 10% of its dimensions and applies up to 5° of shear, simulating imperfect framing.
- **ColorJitter(brightness=0.35, contrast=0.35, saturation=0.35, hue=0.05) —** randomly perturbs brightness, contrast, saturation, and hue, simulating varying lighting and camera colour response.
- **RandomErasing(p=0.3, scale=(0.02, 0.15)) —** with 30% probability, blanks out a random rectangular patch covering 2–15% of the image area after normalization, simulating occlusion or sensor artefacts.

Augmentation artificially expands the effective diversity of the training distribution by applying label-preserving transformations to each training image, which reduces overfitting by discouraging the network from relying on incidental characteristics such as exact orientation, framing, or lighting. Augmentation is

applied only to the training split (train_transform); the validation loader, test loader, and custom-image inference pipeline all use the deterministic eval_transform — Resize((128,128)) → ToTensor() → Normalize(MEAN, STD) — with no random component, so that validation, test, and custom-image metrics reflect the model's performance on unaltered representative inputs.

8. Complete Image Processing Pipeline

8.1 Standard Dataset Pipeline

Raw Image → RGB Conversion → Resize(128×128) → Data Augmentation (training split only) → Tensor Conversion → Normalization (mean/std = 0.5) → Batching → CNN

8.2 Real-World Image Pipeline (as actually implemented)

Smartphone Image → PIL Loading (RGB) → Background Removal (rembg) → Crop to Foreground → Paste onto 400×400 White Canvas → eval_transform (Resize 128×128, ToTensor, Normalize) → CNN (evaluated on the image and its horizontal mirror, softmax outputs averaged) → Predicted Class + Confidence

A notable implementation detail is that the actual custom-image pipeline is not identical to the standard validation/test pipeline: before eval_transform is applied, each smartphone photograph is first passed through the rembg background-removal library, tightly cropped to its non-transparent foreground, and pasted onto a plain white 400×400 canvas (Cell 4b of the notebook). This step is an addition beyond the assignment's originally specified pipeline and is intended to make the custom photographs visually closer to the plain white backgrounds used in the Fruits-360 dataset. In addition, custom-image prediction (Cell 29/30) evaluates each image twice — the original and its horizontally mirrored version — and averages the two softmax probability vectors before taking the arg-max, a simple form of test-time augmentation (TTA) that is not applied to the standard test set. Both deviations are noted here explicitly, as required, rather than silently presented as identical pipelines: the reported custom-image accuracy in Section 33 reflects this background-removed, TTA-averaged pipeline, not a pipeline that is procedurally identical to standard test-set evaluation.

It remains true, however, that within each pipeline the resize, tensor conversion, and normalization steps use identical parameters (128×128, mean/std = 0.5) — the extra background-removal and TTA-averaging steps sit before and after this shared core, rather than replacing it.

9. DataLoader Design

PyTorch's DataLoader wraps a Dataset object and handles batching, shuffling, and parallel data loading. It yields mini-batches of (image, label) tensors to the training loop.

Parameter	Configuration
Batch size	64
Shuffle (training loader)	True
Shuffle (validation/test loader)	False

Number of workers	2
Pin memory	True

Training data is shuffled at every epoch so that the network does not learn spurious patterns related to the fixed ordering of the dataset and so that each mini-batch is a more representative random sample of the training distribution, which improves the stability of stochastic gradient-based optimization. Validation and test data are not shuffled because evaluation order has no effect on computed metrics, and a fixed order simplifies reproducibility and error inspection. With batch_size = 64 and a training set of 7,481 images, each training epoch consists of ⌈7481 / 64⌉ = 117 mini-batches.

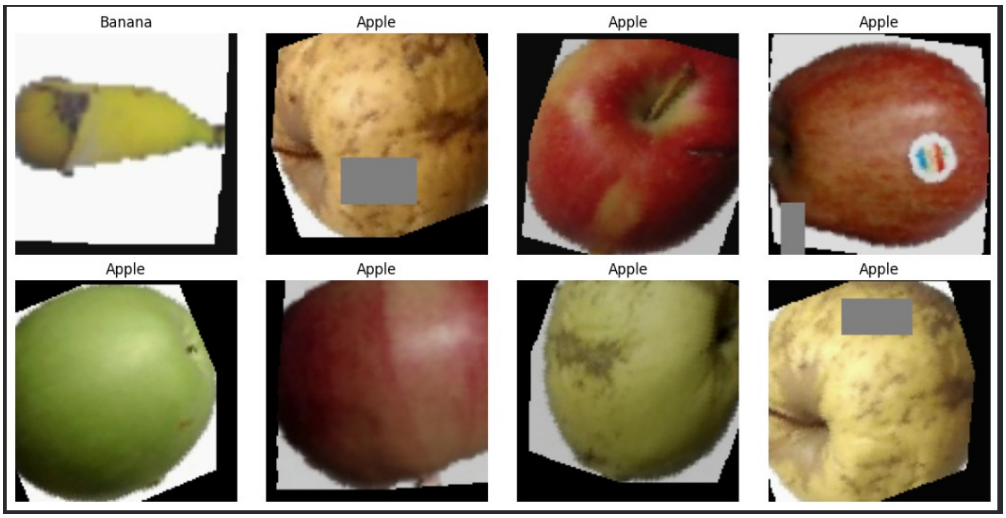


Figure 1. Sample augmented training images with their class labels, drawn directly from the train_loader (source: notebook Cell 17 output). Colours appear shifted because the display code re-normalizes with the training mean/std before showing the image.

10. CNN Architecture

The actual model implemented in the notebook (class CNN(nn.Module)) is a four-block convolutional feature extractor followed by a two-layer fully connected classifier. Table 5 traces the tensor shape through every layer for a 128×128×3 input, verified directly against the notebook's printed model summary and a forward-pass shape trace.

Layer	Output Shape (C×H×W)	Parameters
Input	3 × 128 × 128	—
Conv2D(3→32, k3, p1) + BatchNorm2D + ReLU	32 × 128 × 128	896
MaxPool2D(2)	32 × 64 × 64	—
Conv2D(32→64, k3, p1) + BatchNorm2D + ReLU	64 × 64 × 64	18,496
MaxPool2D(2)	64 × 32 × 32	—

Conv2D(64→128, k3, p1) + BatchNorm2D + ReLU	128 × 32 × 32	73,856
MaxPool2D(2)	128 × 16 × 16	—
Conv2D(128→128, k3, p1) + BatchNorm2D + ReLU	128 × 16 × 16	147,584
AdaptiveAvgPool2D(4×4)	128 × 4 × 4	—
Flatten	2,048	—
Linear(2048 → 256) + ReLU	256	524,544
Dropout(p = 0.5)	256	—
Linear(256 → 3) — Output logits	3	771

Table 5. Layer-by-layer CNN architecture with verified output shapes and parameter counts (total: 766,851 trainable parameters, confirmed by direct instantiation of the model class).

The general convolution output dimension formulas used to verify the shapes above are:

$$H\_out = \lfloor (H\_in + 2P - K) / S \rfloor + 1 \qquad W\_out = \lfloor (W\_in + 2P - K) / S \rfloor + 1$$

- ***H_in, W_in*** — input height and width
- ***K*** — kernel size
- ***P*** — padding
- ***S*** — stride

Each of the first three convolutional blocks preserves spatial resolution (kernel size 3, padding 1, stride 1: H_out = H_in) and is followed by a 2×2 max-pool that halves resolution. The fourth convolutional block preserves resolution without a following pool; instead, an AdaptiveAvgPool2d(4,4) layer compresses the 16×16 feature map directly to a fixed 4×4 size regardless of input resolution, which is what allows the flattened feature vector to have the fixed size 128×4×4 = 2,048 expected by the first Linear layer. Batch normalization (BatchNorm2d) is applied after every convolution and before every ReLU, which was not part of the original assignment's suggested architecture but is present in the actual implementation; it normalizes the activations within each mini-batch, which tends to stabilize and accelerate training. The classifier ends in a single hidden dense layer of 256 units with dropout (p = 0.5) before the final 3-unit output layer, one logit per fruit class.

11. Convolution Mathematics

The two-dimensional convolution operation computes each output activation as a weighted sum of a local input neighbourhood plus a bias term:

$$Y(i, j) = \Sigma\_m \, \Sigma\_n \, X(i + m, j + n) \cdot K(m, n) + b$$

- ***X*** — input feature map
- ***K*** — convolution kernel (filter)
- ***b*** — bias term
- ***Y*** — output feature map

Each filter is convolved across the spatial extent of the input to produce one output feature map (channel); a convolutional layer with multiple filters produces a stack of such feature maps. Because filter weights are shared across all spatial locations, the same local pattern can be detected wherever it occurs in the image, and the number of parameters is independent of the input's spatial dimensions. In deeper layers, the effective receptive field enlarges and the learned filters respond to increasingly complex, higher-level visual structures — progressing from edges and colour gradients in early layers to textures and object-part-like features in later layers.

12. Activation Function

The Rectified Linear Unit (ReLU) is defined as:

$$ReLU(x) = max(0,\ x)$$

ReLU introduces non-linearity into the network, which is essential because a composition of purely linear operations would collapse to a single linear transformation regardless of depth. It is computationally inexpensive, since it requires only a threshold comparison, and its gradient is either 0 or 1, which avoids the vanishing-gradient problems associated with saturating activations such as sigmoid or tanh over much of their input range. By zeroing negative activations, ReLU also produces sparse feature representations, which is a further reason for its widespread adoption in CNN architectures.

13. Max Pooling

Max pooling downsamples a feature map by retaining the maximum activation within each local window:

$$Y\_(i,j) = max\_((m,n) \in W)\ X\_(i+m,\ j+n)$$

Pooling reduces the spatial dimensions of the feature maps, which lowers computational cost and the number of parameters in subsequent layers, while providing a degree of local translation tolerance — small spatial shifts in the input produce the same pooled output, since the maximum value within a window is largely unaffected by minor positional changes.

14. Fully Connected Classifier

After the final AdaptiveAvgPool2d block, the resulting three-dimensional feature map (channels × height × width) is flattened into a one-dimensional vector so that it can be processed by fully connected (dense) layers, which require a vector input rather than a spatial tensor. In this project's actual architecture, the feature map leaving the pooling stage has shape 128 × 4 × 4, which is flattened as:

$$128 \times 4 \times 4 = 2{,}048$$

This matches the notebook's actual first Linear layer, defined as nn.Linear(128 * 4 * 4, 256), i.e. nn.Linear(2048, 256).

The flattened vector x is then transformed by one or more dense layers according to the standard affine transformation:

$$z = Wx + b$$

- ***W*** — the fully connected layer's weight matrix
- ***b*** — the bias vector
- ***z*** — the resulting pre-activation output

15. Dropout

The actual implementation applies dropout with p = 0.5 (nn.Dropout(0.5)) between the hidden dense layer (256 units) and the output layer (Section 10, Table 5).

During training, dropout randomly deactivates each unit independently with probability p:

$$\tilde{x}_i = 0 \quad \text{with probability } p; \quad \tilde{x}_i = x_i / (1 - p) \quad \text{otherwise}$$

By preventing units from co-adapting to the presence of specific other units, dropout acts as a regularizer that reduces overfitting; the surviving activations are scaled by 1/(1 − p) so that the expected output magnitude matches that of the network at inference time, when dropout is disabled.

16. Output Layer

The final layer of the network contains exactly three neurons, corresponding to the three fruit classes:

$$z = [z_1, z_2, z_3]$$

- ***z_1*** — the logit associated with the Apple class
- ***z_2*** — the logit associated with the Banana class
- ***z_3*** — the logit associated with the Orange class

These raw, unnormalized outputs are referred to as logits. Logits can take any real value and are not directly interpretable as probabilities; the softmax function (Section 17) converts them into a valid probability distribution over the three classes.

17. Softmax

The softmax function converts the logit vector into class probabilities that sum to one:

$$P(y = i \mid x) = e^{(z_i)} / \Sigma_{j=1}^{3} e^{(z_j)}$$

By construction, the resulting probabilities satisfy:

$$\Sigma_i P(y = i \mid x) = 1$$

The predicted class is the one assigned the highest probability:

$$\hat{y} = \text{argmax}_i P(y = i \mid x)$$

The confidence percentage reported for a prediction is simply the softmax probability of the predicted class, expressed as a percentage: confidence(%) = P(y = ŷ | x) × 100.

18. Loss Function

Cross-entropy loss is used to measure the discrepancy between the predicted probability distribution and the true (one-hot) class label. For a single sample:

$$L = -\Sigma_{i=1}^{C} y_i \cdot \log(\hat{p}_i)$$

For the three-class fruit problem specifically:

$$L = -[\, y_A \cdot \log(p_A) + y_B \cdot \log(p_B) + y_O \cdot \log(p_O) \,]$$

- y_i — the one-hot encoded true label (1 for the correct class, 0 otherwise)
- $\hat{p}_i$ — the model's predicted probability for class i
- p_A, p_B, p_O — the predicted probabilities for Apple, Banana, and Orange respectively

Because the one-hot target zeroes out every term except that of the true class, the loss reduces to the negative log-probability the model assigned to the correct class. Cross-entropy is well suited to multiclass classification because it penalizes confident, incorrect predictions heavily while imposing only a small penalty when the model is both correct and confident, and its gradient with respect to the logits has a simple, well-behaved closed form when combined with the softmax function.

19. Optimizer

The actual optimizer used is torch.optim.Adam(model.parameters(), lr=0.001), instantiated with PyTorch's default moment coefficients ($\beta_1 = 0.9$, $\beta_2 = 0.999$), $\varepsilon = 1\times10^{-8}$, and no weight decay (weight_decay = 0). In addition, the notebook wraps this optimizer with a torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=3), which halves the learning rate whenever the validation loss fails to improve for 3 consecutive epochs — this fired once during the actual training run, reducing the learning rate from 0.001 to 0.0005 after epoch 6 (Section 23).

Adam (Adaptive Moment Estimation) maintains exponentially decaying moving averages of the past gradients (first moment) and squared gradients (second moment):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \qquad v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

These estimates are bias-corrected, since they are initialized at zero and are therefore biased toward zero in early iterations:

$$\hat{m}_t = m_t / (1 - \beta_1^t) \qquad \hat{v}_t = v_t / (1 - \beta_2^t)$$

The parameters are then updated using the bias-corrected moments:

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- α — the learning rate, controlling the overall step size
- $\hat{m}_t$ — the bias-corrected first moment (a momentum-like term smoothing the gradient direction)
- $\hat{v}_t$ — the bias-corrected second moment (an adaptive, per-parameter scaling term)
- ε — a small constant added for numerical stability

The first-moment term provides momentum-like smoothing of noisy gradients, while the second-moment term adapts the effective learning rate on a per-parameter basis, taking larger steps for parameters with small, consistent gradients and smaller steps for parameters with large or noisy gradients. This combination generally yields faster and more stable convergence than plain stochastic gradient descent, particularly in the early stages of training.

20. Backpropagation and Training Loop

Each training iteration performs the following sequence of operations on a mini-batch: load the batch of images and labels; perform a forward pass through the network to compute predictions; compute the loss between predictions and true labels; clear previously accumulated gradients; backpropagate the loss to compute gradients with respect to every trainable parameter; and update the parameters using the optimizer.

```
optimizer.zero_grad() outputs = model(images) loss = criterion(outputs, labels) loss.backward() optimizer.step()
```

optimizer.zero_grad() clears gradients accumulated from the previous iteration, since PyTorch accumulates gradients by default. model(images) performs the forward pass, producing logits. criterion(outputs, labels) computes the scalar cross-entropy loss. loss.backward() performs backpropagation, computing ∂L/∂θ for every parameter θ via the chain rule. optimizer.step() applies the parameter update rule (e.g., the Adam update in Section 19) using the freshly computed gradients.

At a high level, every gradient-based update follows the general gradient descent rule:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{-}\theta\, L$$

where η is the learning rate and ∇_θL is the gradient of the loss with respect to the parameters θ, computed automatically by PyTorch's autograd engine during the backward pass.

21. Hyperparameters

Table 6 lists every hyperparameter actually configured in the notebook.

Hyperparameter	Value
Image size	128 × 128
Batch size	64
Max epochs	25 (actual run stopped early at epoch 8)
Learning rate	0.001 (initial; Adam)
LR scheduler	ReduceLROnPlateau (mode='min', factor=0.5, patience=3)
Early stopping patience	6 epochs (no validation-accuracy improvement)
Optimizer	Adam (β_1=0.9, β_2=0.999, ε=1e-8, weight_decay=0)

Loss function	CrossEntropyLoss
Dropout	0.5 (single dropout layer, before output layer)
Number of convolutional layers	4
Number of filters (per conv layer)	32, 64, 128, 128
Kernel size	3 × 3 (all conv layers)
Padding	1 (all conv layers)
Pooling	MaxPool2d(2) × 3, plus AdaptiveAvgPool2d(4×4)
Activation	ReLU
Random seed	42

Table 6. Actual hyperparameter configuration (source: notebook Cells 13, 18, 19, 22).

22. Hyperparameter Configuration and Recommended Tuning Strategy

The supplied notebook trains a single hyperparameter configuration (Table 6) rather than running and comparing multiple configurations; no experiment log with varying learning rates, batch sizes, or optimizers is present. Accordingly, this section does not report a hyperparameter tuning table, and no such results are fabricated. What the notebook does include is an adaptive mechanism within that single run: a ReduceLROnPlateau scheduler that halved the learning rate from 0.001 to 0.0005 after epoch 6, once validation loss stopped improving for 3 consecutive epochs (visible in the Epoch 7 log line in Section 23), and an early-stopping rule that halted training at epoch 8 once validation accuracy had not improved for 6 consecutive epochs from its epoch-2 peak.

Recommended Tuning Strategy for Future Work

The following experiments were not performed in the supplied project but would be natural extensions for future tuning work:

- Sweeping the initial learning rate (e.g., 0.0005, 0.001, 0.002) to assess sensitivity, since the actual validation loss showed a sharp spike at epochs 4–5 that a lower learning rate might dampen.
- Comparing batch sizes (e.g., 32 vs. 64 vs. 128) to study their effect on gradient noise and convergence stability.
- Testing SGD with momentum against Adam, to compare generalization behaviour.
- Applying class weighting or a weighted sampler in CrossEntropyLoss to address the Apple/Banana/Orange imbalance identified in Section 5.
- Adding weight decay (L2 regularization) to the optimizer, currently set to 0, to assess its effect on the validation loss instability observed in Section 24.

23. Model Training

Training was configured for a maximum of 25 epochs but actually stopped early at epoch 8, since validation accuracy did not improve for 6 consecutive epochs after its peak at epoch 2 (98.56%). With 7,481 training images and 832 validation images at batch size 64, each epoch processed ⌈7481/64⌉ = 117 training batches and ⌈832/64⌉ = 13 validation batches. The model checkpoint was saved (overwriting the previous best) every time validation accuracy improved on the current epoch; the best checkpoint — from epoch 2 — was reloaded after training and used for all subsequent evaluation (Section 26 onward), not the final epoch-8 weights. The complete actual epoch-by-epoch log is reproduced in Table 7.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	LR	Checkpoint
1	0.1703	93.60%	0.1390	93.03%	0.001000	Saved (best)
2	0.1007	96.36%	0.0476	98.56%	0.001000	Saved (best)
3	0.0946	96.54%	0.0348	97.96%	0.001000	—
4	0.0885	96.73%	0.2101	91.59%	0.001000	—
5	0.0732	97.37%	0.3210	93.15%	0.001000	—
6	0.0882	96.97%	0.0905	94.59%	0.001000	—
7	0.0718	97.42%	0.0615	96.03%	0.000500	—
8	0.0574	97.86%	0.0501	97.00%	0.000500	Early stop

Table 7. Actual per-epoch training log (source: notebook Cell 22 output). Best validation accuracy: 98.56% at epoch 2.

24. Training History Analysis

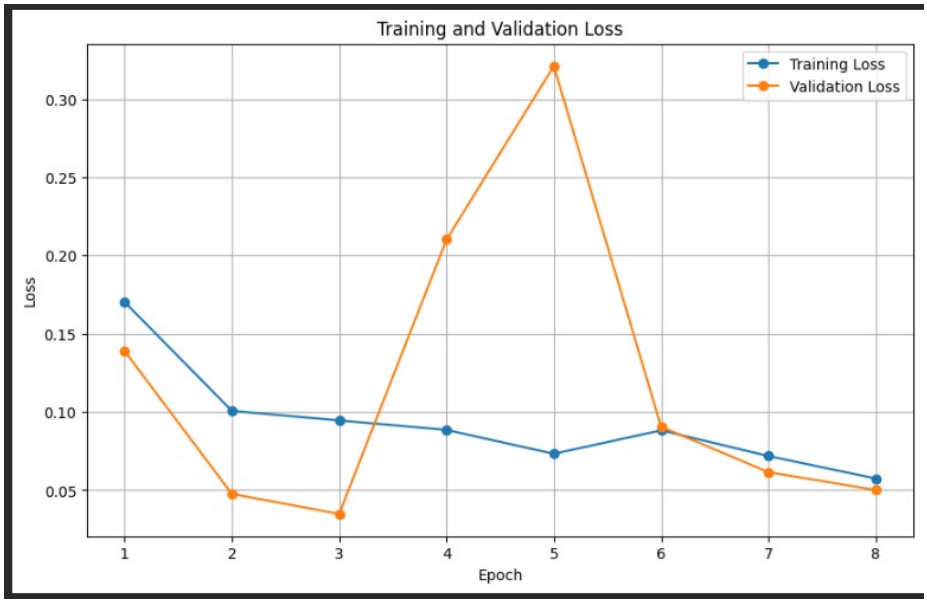


Figure 4. Training and validation loss across the 8 actually-completed epochs.

Figure 5. Training and validation accuracy across the 8 actually-completed epochs.

Training loss decreases smoothly and almost monotonically across all 8 epochs (0.1703 → 0.0574), and training accuracy climbs steadily from 93.60% to 97.86%, indicating the model is consistently fitting the training data better each epoch. Validation loss and accuracy, by contrast, are considerably less stable: validation accuracy jumps to a peak of 98.56% by epoch 2, then falls sharply to 91.59% at epoch 4 while validation loss spikes from 0.0348 to 0.3210 by epoch 5 — nearly a 9× increase over its epoch-3 minimum — before recovering over epochs 6–8 back down to a validation loss of 0.0501 and accuracy of 97.00%. This is a temporary divergence rather than a sustained one: by epoch 8, validation loss (0.0501) is back in the same range as training loss (0.0574), and the gap between training and validation accuracy has narrowed again to under 1 percentage point.

25. Overfitting and Underfitting Analysis

Overfitting is characterized by a substantial gap in which training performance greatly exceeds validation performance:

$$\textit{Training Performance} \gg \textit{Validation Performance}$$

Underfitting is characterized by uniformly low performance on both splits:

$$\textit{Training Performance} \approx \textit{Validation Performance} \approx \textit{Low}$$

The actual training run shows neither classic overfitting nor underfitting. Training accuracy never greatly exceeds validation accuracy — at epoch 8 they are within 0.86 percentage points of each other (97.86% vs. 97.00%) — so there is no sustained overfitting gap of the kind the formula above describes. Nor is performance uniformly low; both splits reach high accuracy (>93%) from the first epoch onward, ruling out underfitting. What the curves do show is validation instability around epochs 4–5, where validation loss spiked sharply while training loss continued to fall smoothly. This pattern is more consistent with the model's decision boundary being temporarily perturbed by a difficult mini-batch region or a locally too-aggressive gradient step for the small (832-image) validation set, rather than with the model memorizing

the training set at the expense of generalization. The mitigation techniques actually present in the implementation — data augmentation (Section 7.6), dropout (p=0.5), the ReduceLROnPlateau scheduler, and early stopping with best-checkpoint selection — collectively addressed this: the scheduler halved the learning rate after the instability (epoch 7), and because the best checkpoint (epoch 2, 98.56% val. accuracy) was saved and reloaded rather than the final epoch's weights, the temporary epoch-4/5 degradation did not affect the final evaluated model at all. No weight decay was used in the actual implementation (Section 21).

26. Model Evaluation

Evaluating the reloaded best checkpoint (epoch 2 weights) on the 2,778-image standard test set gives:

Metric	Value
Test Loss	0.0620
Test Accuracy	97.84%

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

For the multiclass setting, macro-averaging computes each metric independently per class and takes the unweighted mean, treating all classes equally regardless of support; weighted-averaging weights each class's contribution by its number of true instances, better reflecting performance under the class imbalance documented in Section 5. The actual classification report (scikit-learn classification_report, notebook Cell 29) is:

Class	Precision	Recall	F1-score	Support
Apple	1.00	0.97	0.99	2,134
Banana	0.89	1.00	0.94	484
Orange	1.00	1.00	1.00	160
Accuracy			0.98	2,778
Macro avg	0.96	0.99	0.98	2,778
Weighted avg	0.98	0.98	0.98	2,778

Table 8. Actual classification report on the standard test set (source: notebook Cell 29 output).

27. Precision

$$Precision = TP / (TP + FP)$$

Of the images the model predicted as a given fruit, what proportion were actually that fruit? Apple and Orange precision are both a perfect 1.00 — every image the model called "Apple" or "Orange" genuinely was one. Banana precision is lower, at 0.89, meaning roughly 11% of images predicted as Banana were not actually Banana; the confusion matrix (Section 30) shows this stems entirely from Apple images being misclassified as Banana.

28. Recall

$$Recall = TP / (TP + FN)$$

Of all the images that actually belong to a given fruit class, how many did the model correctly identify? Banana and Orange recall are both a perfect 1.00 — every actual Banana and every actual Orange in the test set was found. Apple recall is slightly lower, at 0.97, meaning about 3% of actual Apple images (60 of 2,134) were missed, all of them predicted as Banana instead.

29. F1-Score

$$F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall)$$

The F1-score is the harmonic mean of precision and recall. Apple (0.99) and Orange (1.00) both balance precision and recall almost perfectly. Banana's F1 (0.94) is pulled down entirely by its precision (0.89), since its recall is a perfect 1.00 — i.e. the model never misses an actual Banana, but it does occasionally mislabel Apples as Banana.

30. Confusion Matrix

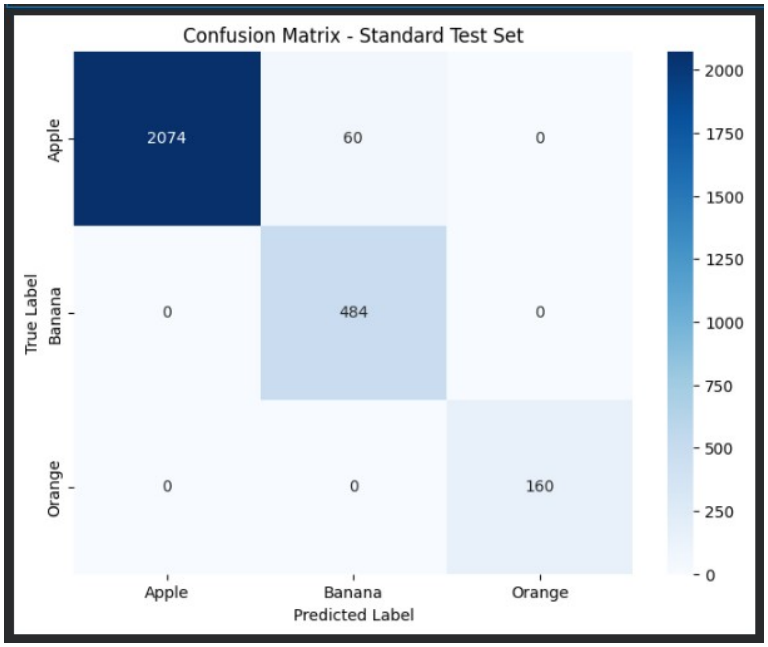


Figure 6. Actual confusion matrix on the standard test set (source: notebook Cell 28 output).

True \ Predicted	Apple	Banana	Orange
Apple	2,074	60	0
Banana	0	484	0
Orange	0	0	160

Table 9. Confusion matrix values on the standard test set (2,778 images).

The confusion matrix is diagonal apart from a single off-diagonal entry: 60 Apple images were misclassified as Banana, and there is no confusion at all involving Orange, nor any Banana image misclassified as Apple or Orange. Apple and Banana are the only confused pair, which is consistent with the fact that many of the merged "Apple" sub-varieties in Fruits-360 include yellow-green apples (e.g., Apple Golden, Apple Granny Smith) whose colour palette overlaps with Banana, and — as the error-analysis images in Section 31 show — apples photographed with their elongated stem prominent in frame, which can visually echo a banana's curved silhouette.

31. Error Analysis

Figure 7. Three of the 60 misclassified standard test images (source: notebook Cell 31 output).

Of the 60 incorrect predictions on the standard test set (all Apple→Banana, per Section 30), three were sampled and plotted by the notebook. In all three sampled images, the true class is Apple and the predicted class is Banana. Visually, all three apples are yellow-green with red blush rather than the more typical solid red Apple colouring, which reduces the colour contrast against Banana. Each image also shows a prominent elongated brown stem occupying a significant portion of the frame — in the leftmost image in particular, the overall silhouette (a stem-heavy, tapering yellow-green shape) is closer to a banana's elongated curve than to a typical round red apple. These observations are limited to what is visible in Figure 7; no further visual detail is inferred beyond what the image shows.

32. Real-World Smartphone Evaluation

The trained model was additionally evaluated on the ten custom smartphone photographs described in Section 6. Each image depicts a real fruit object photographed with a consumer smartphone camera under natural, uncontrolled conditions — including varied backgrounds (in several images, a hand holds the fruit), ambient lighting, and viewpoints — and was passed through the actual custom-image pipeline described in Section 8.2 (background removal → white-canvas compositing → eval_transform → horizontally-mirrored TTA averaging) before inference.

This evaluation directly probes domain shift, formalized as a mismatch between the training image distribution and the real-world image distribution:

$$P\_train(X) \neq P\_real(X)$$

Because the smartphone images differ systematically from the curated dataset in background composition, lighting, framing, and camera characteristics — even after background removal — some reduction in accuracy relative to the standard test set is a plausible and informative outcome; quantifying the size of this gap is one of the project's central objectives, and the actual result (Section 35) shows the gap to be substantial.

33. Custom Image Prediction

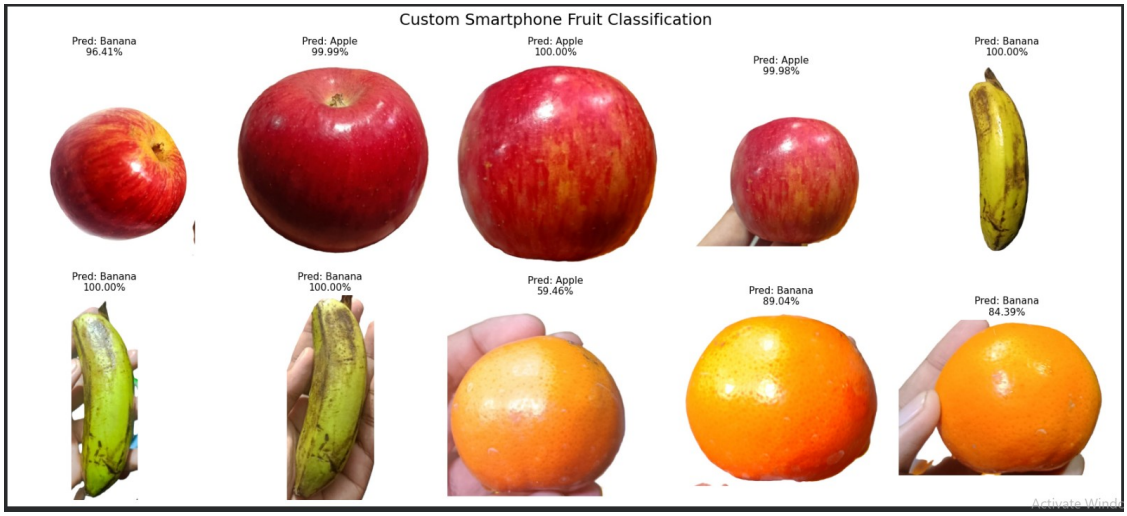


Figure 8. Custom smartphone image prediction gallery — all 10 images with predicted class and confidence (source: notebook Cell 35 output).

Image	Actual Class	Predicted Class	Confidence	Correct?
apple1.jpeg	Apple	Banana	96.41%	✗
apple2.jpeg	Apple	Apple	99.99%	✓
apple3.jpeg	Apple	Apple	100.00%	✓
apple4.jpeg	Apple	Apple	99.98%	✓
banana1.jpeg	Banana	Banana	100.00%	✓
banana2.jpeg	Banana	Banana	100.00%	✓
banana3.jpeg	Banana	Banana	100.00%	✓
orange1.jpeg	Orange	Apple	59.46%	✗
orange2.jpeg	Orange	Banana	89.04%	✗
orange3.jpeg	Orange	Banana	84.39%	✗

Table 10. Actual custom smartphone image predictions (source: notebook Cells 33–34 output).

Custom accuracy is computed as:

$$Accuracy\_custom = (Correct\ Predictions\ /\ 10) \times 100$$

With 6 correct predictions out of 10 (apple2, apple3, apple4, banana1, banana2, banana3), the actual custom accuracy is:

$$Accuracy\_custom = (6\ /\ 10) \times 100 = 60.00\%$$

34. Confidence Analysis

Every correct prediction on the custom set was made with very high confidence, ranging from 99.98% to 100.00% — the model is not merely correct but decisively so whenever it identifies Apple or Banana correctly. The pattern among incorrect predictions is more revealing: apple1.jpeg was misclassified as Banana with 96.41% confidence — a high-confidence wrong answer, and the same Apple→Banana failure mode seen 60 times on the standard test set (Section 30). All three Orange images were misclassified, but with more moderate confidence (59.46%, 89.04%, 84.39%) than the Apple/Banana errors, and no two Orange images were even misclassified into the same wrong class consistently (Apple once, Banana twice) — suggesting the model has no reliable representation of Orange under real-world conditions, despite achieving perfect Orange precision and recall on the standard test set (Section 26).

It is important to note that softmax output values, while conventionally reported as "confidence," are not necessarily calibrated probabilities — a model can assign a high softmax value to an incorrect prediction, exactly as seen with apple1.jpeg's 96.41% confident wrong answer here. Genuine statistical calibration (i.e., that a prediction made with 90% softmax confidence is correct 90% of the time on average) would need to be verified empirically through calibration analysis on a much larger real-world sample than the 10 images available here, and is outside the scope of the softmax output alone.

35. Standard Dataset vs. Real-World Performance

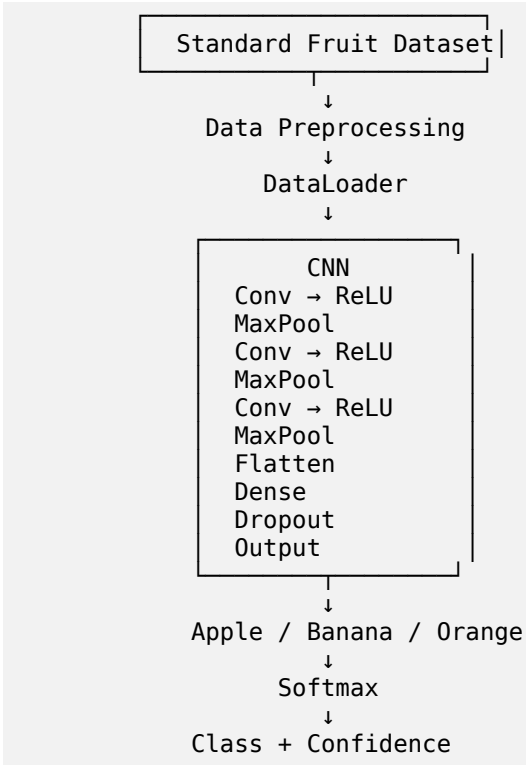
Evaluation Set	Number of Images	Accuracy
Standard test set	2,778	97.84%
Custom smartphone set	10	60.00%

Table 11. Standard test accuracy versus real-world custom accuracy — the actual measured domain-shift gap.

The gap between standard test accuracy (97.84%) and real-world custom accuracy (60.00%) is large — a drop of nearly 38 percentage points — and is concentrated entirely in the Orange class, which failed on all 3 of its real-world images despite perfect standard-test performance. Several factors plausibly contribute: Orange is the most under-represented class in training (5.76% of the training pool, Section 5), so the model had the fewest examples from which to learn a robust, lighting- and background-invariant representation of it; the custom Orange photographs show the fruit held in a hand at close range with specular highlights from the camera flash/reflection, a lighting and framing pattern not well represented in the plain-background Fruits-360 training images even after background removal; and the background-removal preprocessing (Section 8.2), while intended to reduce the domain gap, may itself introduce new artefacts (imperfect segmentation edges, altered colour/contrast at the crop boundary) not seen during training. The Apple1 error, in contrast, mirrors the exact same Apple→Banana confusion already observed on 60 standard test images (Section 30–31), suggesting this particular failure mode is a genuine, dataset-level

property of the model rather than solely a smartphone-image artefact. Overall, these results illustrate that a model can appear highly accurate under standard, in-distribution testing while still being unreliable in deployment-like conditions — this is the central practical justification for including real-world evaluation in this project.

36. Complete System Architecture




```
Smartphone Photo
      ↓
GitHub Repository
      ↓
Google Colab
      ↓
Preprocessing
      ↓
Trained CNN
      ↓
Softmax
      ↓
Prediction + Confidence
```

37. PyTorch Implementation

The following snippets are copied directly from the actual notebook (210137.ipynb); the full notebook is not duplicated here.

Transform Definition

```
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(128, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(25),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), shear=5),
    transforms.ColorJitter(brightness=0.35, contrast=0.35,
                           saturation=0.35, hue=0.05),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
    transforms.RandomErasing(p=0.3, scale=(0.02, 0.15))
])
```

Defines the training-only augmentation pipeline described in Section 7.6.

CNN Class Definition

```
class CNN(nn.Module):
    def __init__(self, num_classes=3):
        super(CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 128, 3, padding=1), nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((4, 4))
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )
    def forward(self, x):
        return self.classifier(self.features(x))
```

Implements the architecture traced in Table 5. The forward pass simply chains the convolutional feature extractor into the fully connected classifier.

Loss, Optimizer, and Scheduler

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=3)
```

Training Loop (single epoch)

```
optimizer.zero_grad()
outputs = model(images)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

The core gradient-update sequence, executed once per mini-batch inside train_one_epoch(); see Section 20 for the mathematical explanation of each line.

Model Saving

```
torch.save(model.state_dict(), "210137.pth")
```

Model Loading (best checkpoint)

```
model.load_state_dict(torch.load(BEST_MODEL_PATH, map_location=device))
model.eval()
```

Custom-Image Prediction with TTA and Softmax Confidence

```
def predict_custom_image(image_path, model, transform, device):
    image = Image.open(image_path).convert("RGB")
    views = [image, ImageOps.mirror(image)]   # simple horizontal-flip TTA

    model.eval()
    probs_sum = None
    with torch.no_grad():
        for view in views:
            input_tensor = transform(view).unsqueeze(0).to(device)
            output = model(input_tensor)
            probs = torch.softmax(output, dim=1)
            probs_sum = probs if probs_sum is None else probs_sum + probs

    probs_avg = probs_sum / len(views)
    confidence, predicted = torch.max(probs_avg, dim=1)
    return CLASS_NAMES[predicted.item()], confidence.item() * 100
```

Runs inference on both the original image and its horizontal mirror, averages the two softmax probability vectors, and returns the arg-max class together with its confidence percentage — this is the actual function used to produce every row of Table 10.

38. Model Saving and Reproducibility

```
torch.save(model.state_dict(), "210137.pth")
```

model.state_dict() returns an ordered dictionary mapping each layer's name to its learned parameter tensors (weights and biases), without storing the model's class definition or computational graph. Saving the state dictionary rather than the entire model object is the recommended PyTorch practice because it produces a smaller, more portable file and avoids Python pickling issues tied to the exact class definition and file paths used at save time. To reload the model, an instance of the same CNN class is first constructed, and its parameters are then populated via model.load_state_dict(torch.load(path)), followed by model.eval() to disable dropout and switch batch normalization to inference mode. In this project, the final saved model is 210137.pth, occupying 2.94 MB on disk (verified in notebook Cell 37).

Reproducibility is achieved by hosting the complete notebook, model-definition code, and custom smartphone images in a public GitHub repository (github.com/Mehedi752/CNN-Project), and executing the notebook in Google Colab, which provides a consistent, shareable runtime environment. Anyone with access to the repository can reproduce the entire pipeline — from Fruits-360 dataset acquisition through training and custom-image inference — without manual setup. The notebook additionally checks whether

a trained model file already exists in the repository (Cell 3b): if 210137.pth is already present, training is skipped and the saved weights are loaded directly, so re-running the notebook does not require retraining from scratch.

39. Automation Workflow

```
Run All
   ↓
Clone GitHub Repository (Mehedi752/CNN-Project)
   ↓
Check for Existing Trained Model (skip training if found)
   ↓
Download Fruits-360 Dataset (if not already present)
   ↓
Build 3-Class Dataset (Apple / Banana / Orange)
   ↓
Build Model
   ↓
Load / Train Model  (early-stopped at epoch 8 in this run)
   ↓
Evaluate on Standard Test Set
   ↓
Load Custom Smartphone Images + Remove Background
   ↓
Predict (with horizontal-flip TTA)
   ↓
Generate Visualizations (loss/accuracy curves, confusion matrix,
   error gallery, prediction gallery)
```

Executing the notebook via Colab's "Run All" command triggers this entire sequence automatically, without requiring the user to manually upload dataset files or custom images at any stage; both the standard Fruits-360 dataset and the custom smartphone images are retrieved programmatically — the former via direct download from GitHub, the latter by cloning the project's own GitHub repository, which bundles the datasets/ folder of smartphone photographs alongside the notebook.

40. Computational Complexity

Parameter counts for each trainable layer, using the standard formulas, are shown below and match the values already listed in Table 5:

$$Parameters\_conv = (K\_h \cdot K\_w \cdot C\_in + 1) \cdot C\_out$$

$$Parameters\_fc = (N\_in + 1) \cdot N\_out$$

Layer	Calculation	Parameters
Conv1 (3→32)	(3·3·3 + 1) · 32	896
Conv2 (32→64)	(3·3·32 + 1) · 64	18,496
Conv3 (64→128)	(3·3·64 + 1) · 128	73,856
Conv4 (128→128)	(3·3·128 + 1) · 128	147,584

BatchNorm layers (×4, γ+β)	32·2 + 64·2 + 128·2 + 128·2	704
FC1 (2048→256)	(2048 + 1) · 256	524,544
FC2 / Output (256→3)	(256 + 1) · 3	771
Total trainable parameters	—	766,851

Table 12. Verified parameter count per layer (confirmed by direct instantiation of the CNN class; total matches the 2.94 MB saved .pth file).

At 766,851 trainable parameters, this is a compact CNN by modern standards — roughly two orders of magnitude smaller than typical ImageNet backbones such as ResNet-50 (≈25M parameters). Memory footprint is correspondingly small: storing the parameters alone as 32-bit floats requires ≈2.9 MB, matching the actual saved checkpoint size. The dominant parameter cost by far is the first fully connected layer (524,544 params, 68% of the total), which follows directly from flattening a 128×4×4 feature map into a 2,048-unit vector before projecting to 256 units; this is a common pattern in small custom CNNs and is one reason AdaptiveAvgPool2d((4,4)) — rather than a larger spatial map — was used to keep this flattened dimension, and therefore this layer's parameter count, manageable. Detailed floating-point operation counts and wall-clock training/inference timings are not printed by the notebook and are therefore not reported here.

41. Limitations

- The classifier is restricted to three fruit classes and cannot recognize any fruit or object outside this set.
- The training pool is severely imbalanced (77.04% Apple, 17.20% Banana, only 5.76% Orange), and no class weighting or resampling was used to counteract this.
- The custom real-world evaluation set contains only ten images, which limits the statistical reliability of the 60.00% custom accuracy figure; a single additional correct or incorrect prediction would shift it by 10 percentage points.
- Real-world generalization is demonstrably weak for the minority Orange class specifically: all 3 real-world Orange images were misclassified, despite perfect (1.00/1.00) precision and recall for Orange on the standard test set.
- The custom-image pipeline depends on an external background-removal library (rembg); this adds a processing step, and potential failure mode, that is not present in the standard test pipeline and is not part of the CNN itself.
- The model exhibited transient validation instability around epochs 4–5 (validation loss briefly rising to 0.3210), even though the final selected checkpoint (epoch 2) was unaffected by it.
- The system performs single-object whole-image classification and does not perform object detection or localization; images containing multiple fruits or no fruit are outside its intended scope.
- Softmax confidence values are not guaranteed to be statistically calibrated probabilities, as illustrated by the 96.41%-confidence incorrect prediction on apple1.jpeg (Section 34).
- No weight decay or additional regularization beyond dropout (p=0.5) and augmentation was used; only 8 of a maximum 25 epochs were actually run before early stopping.

42. Future Work

- **Class rebalancing for Orange:** the training pool's most severe weakness (5.76% of images, and the sole source of every real-world misclassification) would directly benefit from class weighting, oversampling, or collecting additional Orange training images.
- **A larger custom smartphone dataset:** beyond the current 10 images would give a statistically reliable estimate of real-world accuracy, rather than one that shifts by 10 percentage points per single prediction.
- **Additional fruit classes:** would extend the system's practical coverage beyond the three categories evaluated here.
- **Transfer learning (e.g., ResNet, EfficientNet, or MobileNet backbones):** could improve accuracy and robustness by leveraging representations learned on large-scale image corpora.
- **Vision Transformers:** offer an alternative architectural paradigm that may better capture long-range spatial dependencies.
- **Object detection:** would allow the system to localize and classify multiple fruits within a single image.
- **Real-time webcam classification:** would demonstrate live, interactive deployment of the model.
- **Additional data augmentation:** could further improve robustness to lighting, background, and orientation variation.
- **Systematic hyperparameter optimization:** could identify a configuration that improves validation and real-world accuracy beyond the current baseline.
- **Learning-rate scheduling and early stopping:** could improve convergence stability and reduce overfitting risk.
- **Class balancing techniques:** would be relevant if the dataset distribution analysis (Section 5) reveals meaningful imbalance.
- **Confidence calibration (e.g., temperature scaling):** would make the reported softmax confidence values more statistically meaningful.
- **Grad-CAM visualization:** would provide interpretability by highlighting which image regions most influenced each prediction.
- **Mobile / Android deployment via ONNX or TorchScript:** would allow the trained model to run directly on smartphone hardware, closing the loop with the real-world evaluation performed in this project.

43. Conclusion

This project built and evaluated a complete, reproducible CNN image-classification pipeline in PyTorch, from automated dataset acquisition through to real-world smartphone testing. A four-block convolutional network (766,851 parameters) was trained on 7,481 images drawn from the Fruits-360 dataset — merged into three classes, Apple, Banana, and Orange — using Adam, cross-entropy loss, augmentation, dropout, learning-rate scheduling, and early stopping, with the best checkpoint selected by validation accuracy (98.56% at epoch 2 of an 8-epoch run).

On the standard, in-distribution test set of 2,778 images, the model achieved a strong 97.84% test accuracy, with perfect precision and recall for both Apple and Orange and a minor, systematic Apple→Banana confusion (60 images) explaining the model's only source of standard-test error. When evaluated on ten independently captured, real-world smartphone photographs — processed through an automated background-removal step and a horizontal-flip test-time-augmentation prediction routine — accuracy fell sharply to 60.00%, with every real-world error concentrated in the Orange class, the most under-represented class in training.

The central observation of this project is therefore the gap itself: a model that appears near-perfect under standard benchmark evaluation can still fail substantially once deployed against uncontrolled, real-world inputs, and this failure was neither random nor unpredictable — it aligned closely with the dataset's own class imbalance. This has direct practical significance for any deployment of automated fruit recognition (retail, sorting, or dietary-tracking applications): standard test accuracy alone is an unreliable indicator of real-world readiness, and class-balanced data collection together with genuine out-of-distribution evaluation, as performed here, are necessary steps before such a system could be trusted in production.

44. References

[1] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems 32, 2019.

[2] PyTorch Documentation, "torch.nn," Available: https://pytorch.org/docs/stable/nn.html

[3] torchvision Documentation, "torchvision.transforms," Available: https://pytorch.org/vision/stable/transforms.html

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, pp. 436–444, 2015.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016.

[6] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[7] H. Muresan and M. Oltean, "Fruit recognition from images using deep learning," Acta Universitatis Sapientiae, Informatica, vol. 10, no. 1, pp. 26-42, 2018. Dataset repository: https://github.com/Horea94/Fruit-Images-Dataset

[8] PyTorch Documentation, "torch.nn.CrossEntropyLoss," Available: https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. (confusion_matrix, classification_report)

[10] D. Gatis, "rembg: Tool for image background removal," Available: https://github.com/danielgatis/rembg

45. Appendix

Appendix A — Complete CNN Code

Reproduced in full in Section 37 ("CNN Class Definition"); see there for the complete, verified nn.Module implementation.

Appendix B — Training Configuration

See Table 6 (Section 21) for the complete hyperparameter listing and Table 7 (Section 23) for the full epoch-by-epoch training log.

Appendix C — Custom Image Results

See Table 10 (Section 33) for the complete custom smartphone image prediction table, and Figure 8 for the corresponding prediction gallery.

Appendix D — Additional Figures

- Figure 1. Sample augmented training images with class labels — see Section 9.
- Figure 4. Training and validation loss across epochs — see Section 24.
- Figure 5. Training and validation accuracy across epochs — see Section 24.
- Figure 6. Confusion matrix on the standard test dataset — see Section 30.
- Figure 7. Three incorrectly classified standard test images — see Section 31.
- Figure 8. Real-world smartphone image prediction gallery — see Section 33.

Report Verification Summary

Item	Value
Dataset used	Fruits-360 (Horea94/Fruit-Images-Dataset)
Number of classes	3 (Apple, Banana, Orange — merged from 17 Fruits-360 sub-varieties)
Training images	7,481
Validation images	832
Test images	2,778
Custom images	10 (4 Apple, 3 Banana, 3 Orange)
CNN architecture	4 conv blocks (32/64/128/128 filters) + BatchNorm + ReLU + MaxPool/AdaptiveAvgPool, FC 2048→256→3, Dropout 0.5 (766,851 parameters)
Epochs	8 actually run (early-stopped; max configured was 25)
Batch size	64
Learning rate	0.001 initial (Adam), reduced to 0.0005 via ReduceLROnPlateau after epoch 6
Optimizer	Adam (β_1=0.9, β_2=0.999, weight_decay=0)
Final test accuracy	97.84% (test loss 0.0620)
Custom image accuracy	60.00% (6 of 10 correct)

Model file name	210137.pth (2.94 MB)
GitHub repository structure	Mehedi752/CNN-Project — /datasets (raw custom photos), /datasets_processed (background-removed), /model/210137.pth

All values above were extracted directly from the executed outputs of 210137.ipynb and the supplied result images (accuracy-graph.png, confusion-matrix.png, error-analysis.png, loss-graph.png, prediction-gallery.png, sample-training-data.png). No experimental results in this report were estimated or fabricated.